

Deconvolution GUI

Program Description and Numerical Methods

Version 0.108.0

<https://github.com/rkotynski/dekonwolucje>

This document describes the graphical workflow, numerical data model, convolution operators, implemented deconvolution methods, quality criteria, automatic parameter tuning, and software architecture of the **dekonwolucje** project. The software is distributed under the MIT License. Source-code comments and internal identifiers remain in English independently of the selected English or Polish GUI language.

August 2026

Contents

1	Purpose and scope	2
2	Forward model and boundary conditions	2
2.1	Zero-boundary linear convolution	2
2.2	Circular FFT model	3
3	Committed calculation image and PSF	3
3.1	Source arrays and reset	3
3.2	Image and PSF floors	4
3.3	Rectangular PSF selection and normalization	4
3.4	Automatic initial support	4
4	Graphical workflow	4
4.1	Tab 1: image and PSF	4
4.2	Tab 2: thresholds and calculation PSF	5
4.3	Tab 3: algorithm	6
4.4	Tab 4: results	6
5	Wiener filtering	7
6	Richardson–Lucy family	7
6.1	Classical Richardson–Lucy	7
6.2	Richardson–Lucy–Wiener	7
6.3	Richardson–Lucy–Rosen	7
7	Landweber, Kaczmarz, and Adam methods	8
7.1	Landweber	8
7.2	Block Kaczmarz	8
7.3	PyTorch Adam TV-MAP	11
8	Blind deconvolution	11
8.1	Blind Richardson–Lucy	11
8.2	Blind Adam TV-MAP	12
9	Metrics and best-iteration selection	12
10	Auto tuning and cancellation	12
11	Batch processing and CUDA	12
12	Python API and use without the GUI	12
12.1	Public objects and functions	12
12.2	Automatic parameter selection	14
12.3	Standalone examples	15

12.4 Advanced execution hooks	16
13 Software architecture	16
14 Algorithm registry	16
15 Installation, testing, and extension	17
16 Selected primary and classical references	18
17 Limitations and interpretation	18
18 License and project information	18
19 AI-assisted development disclosure	19

1 Purpose and scope

The program is a desktop research application for restoration of grayscale images blurred by a point-spread function (PSF) and contaminated by noise. It supports non-blind deconvolution with a supplied PSF and blind deconvolution in which both the latent image and the PSF are updated. The GUI separates source-data preparation, committed PSF selection, algorithm parameters, and result assessment.

The same committed calculation image and calculation PSF are used by all algorithms, synthetic degradation, reblur metrics, and blind-PSF initialization. NumPy/SciPy methods and PyTorch variants are available. PyTorch uses `float32` by default and can use CUDA when requested and available.

2 Forward model and boundary conditions

Let $f[m, n]$, $0 \leq m < M$, $0 \leq n < N$, be the unknown image, let $h[u, v]$, $0 \leq u < M_h$, $0 \leq v < N_h$, be the calculation PSF, let g be the measured image, and let n denote additive noise. The committed PSF is nonnegative and normalized so that $\sum_{u,v} h[u, v] = 1$.

For clarity, $*$ denotes convolution rather than pointwise multiplication. The full zero-boundary linear convolution used by the program is defined by

$$(h *_{\text{lin}} f)[r, s] = \sum_{u=0}^{M_h-1} \sum_{v=0}^{N_h-1} h[u, v] \tilde{f}[r-u, s-v],$$

where $\tilde{f}[a, b] = f[a, b]$ inside the image and $\tilde{f}[a, b] = 0$ outside it. Its full result has size $(M + M_h - 1) \times (N + N_h - 1)$. The default image-sized forward operator is the centered *same* crop

$$\mathcal{H}_{\text{lin}} f = C_{\text{same}}(h *_{\text{lin}} f), \quad (C_{\text{same}} y)[m, n] = y[m + s_y, n + s_x],$$

with $s_y = \lfloor (M_h - 1)/2 \rfloor$ and $s_x = \lfloor (N_h - 1)/2 \rfloor$. This is the convention of `scipy.signal.fftconvolve(..., mode="same")`; it is symmetric for odd kernels and intentionally one-sample asymmetric for even kernels.

The compact notation $\mathcal{H}f$ therefore means “convolve f with the PSF h using the boundary and cropping convention assigned to $\mathcal{H}$ ”. It is matrix–vector notation for a linear operator, not a pointwise product. The discrete observation model is

$$g = \mathcal{H}f + n. \tag{1}$$

The project deliberately distinguishes two choices of $\mathcal{H}$.

2.1 Zero-boundary linear convolution

Most iterative methods and synthetic disturbance generation use $\mathcal{H} = \mathcal{H}_{\text{lin}}$ as defined above. The FFT implementation evaluates the full linear convolution on a canvas of size

$$(M + M_h - 1) \times (N + N_h - 1). \tag{2}$$

The discrete complex inner product used here is

$$\langle x, y \rangle = \sum_{m,n} \overline{x[m, n]} y[m, n].$$

The adjoint $\mathcal{H}^*$ is defined by

$$\langle \mathcal{H}x, y \rangle = \langle x, \mathcal{H}^* y \rangle.$$

If A is the matrix representation of $\mathcal{H}$, then $\mathcal{H}^*$ is represented by the Hermitian transpose $A^* = \overline{A}^\top$. Because the spatial PSF and the crop/embedding operations used here are real, this reduces to the ordinary transpose $A^\top$, but it is still the adjoint of the complete forward operator, not merely the elementwise conjugate of the PSF samples. The corresponding adjoint $\mathcal{H}_{\text{lin}}^*$ is therefore the transpose of the exact zero-boundary convolution followed by the centered crop, including the asymmetric convention needed for even kernels.

2.2 Circular FFT model

The closed-form Wiener inverse uses circular convolution because this operator is diagonal in the two-dimensional discrete Fourier basis. NumPy/SciPy and PyTorch are used with their default `norm="backward"` convention. For an $M \times N$ array x , the transform pair is

$$X[k, \ell] = \mathcal{F}_{M,N}\{x\}[k, \ell] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x[m, n] \exp\left[-2\pi i \left(\frac{km}{M} + \frac{\ell n}{N}\right)\right],$$

$$x[m, n] = \mathcal{F}_{M,N}^{-1}\{X\}[m, n] = \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{\ell=0}^{N-1} X[k, \ell] \exp\left[2\pi i \left(\frac{km}{M} + \frac{\ell n}{N}\right)\right].$$

Thus the forward DFT is unnormalized and the factor $1/(MN)$ occurs only in the inverse DFT. With this pair, neither linear nor circular convolution needs an extra numerical scale factor. In particular, if $P = M + M_h - 1$, $Q = N + N_h - 1$, and $f_{P,Q}$ and $h_{P,Q}$ are the two arrays zero-padded to the full linear-convolution canvas, then

$$h *_{\text{lin}} f = \mathcal{F}_{P,Q}^{-1}\{\mathcal{F}_{P,Q}\{h_{P,Q}\} \mathcal{F}_{P,Q}\{f_{P,Q}\}\}.$$

The implementation may use the equivalent real-input pair `rfft2/irfft2`; it has the same normalization.

To construct the circular PSF array h_0 , the compact calculation PSF is zero-embedded in an $M \times N$ array and rolled by $-[M_h/2]$ and $-[N_h/2]$. This places the kernel-array index $([M_h/2], [N_h/2])$ at the FFT origin, exactly as in the program. Circular convolution is then

$$(h_0 \circledast f)[m, n] = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} h_0[u, v] f[(m-u) \bmod M, (n-v) \bmod N],$$

and it satisfies $\mathcal{F}\{h_0 \circledast f\} = \mathcal{F}\{h_0\} \mathcal{F}\{f\}$. With

$$G = \mathcal{F}\{g\}, \quad H = \mathcal{F}\{h_0\}, \quad (3)$$

where h_0 is the zero-embedded, origin-shifted PSF array defined above. Here H is an $M \times N$ array of complex Fourier coefficients, not the spatial-domain matrix representation of $\mathcal{H}$. For any complex array Z , the notation Z^* means elementwise complex conjugation,

$$Z^*[k, \ell] = \overline{Z[k, \ell]}, \quad |Z|^2 = Z^* Z,$$

where the final product is pointwise. Equivalently, if circular convolution is represented in the Fourier basis by the diagonal matrix $\text{diag}(H)$, its Hermitian adjoint is $\text{diag}(H^*)$. Thus the spectral conjugate H^* is the Fourier-domain representation of the adjoint circular-convolution operator; it should not be confused with taking only an elementwise conjugate of the full spatial operator matrix.

$$\mathcal{H}_{\text{circ}} f = \text{Re} \left\{ \mathcal{F}^{-1}[H \mathcal{F}\{f\}] \right\}. \quad (4)$$

The real part discards only the small imaginary round-off residual expected for real input arrays.

Circular convolution wraps samples across opposite borders: signal leaving the right or lower edge re-enters at the left or upper edge. Zero-boundary linear convolution instead assumes no signal outside the image and discards the outer part when the *same* crop is taken.

Linear and circular boundary models are not identical. A Wiener result can differ near the borders from an iterative result even when both use the same numerical PSF.

3 Committed calculation image and PSF

3.1 Source arrays and reset

The program retains the image and PSF loaded from disk or generated in the GUI as source arrays. **Reset thresholds / PSF selection** restores these sources. In parallel it stores one committed pair: the thresholded calculation image and the thresholded, cropped, nonnegative, unit-sum calculation PSF.

3.2 Image and PSF floors

For nonnegative data x with maximum m and floor T , the applied mapping is

$$x_T = \begin{cases} 0, & x \leq T, \\ m \frac{x - T}{m - T}, & x > T, \end{cases} \quad (5)$$

for $0 < T < m$. Thus the background is removed and the surviving range is stretched to the original maximum. The PSF floor is specified as a fraction q of the source PSF peak, $T_p = qp_{\max}$.

3.3 Rectangular PSF selection and normalization

After thresholding, all full-array samples outside the accepted red rectangle are set to zero. A compact kernel of height M_h and width N_h is extracted around the selected center. Missing samples are zero-padded when the rectangle extends beyond the source array. The calculation PSF is

$$h[u, v] \leftarrow \frac{\max(h_{\text{crop}}[u, v], 0)}{\sum_{u, v} \max(h_{\text{crop}}[u, v], 0)}, \quad (6)$$

so that $h \geq 0$ and $\sum h = 1$.

Pending slider and rectangle changes do not alter the histogram or calculation data until **Apply thresholds / PSF selection now** is pressed. The rectangle can be moved, resized by edges or corners, or scaled about its center with the mouse wheel.

3.4 Automatic initial support

The initial support suggestion estimates the border median b and a robust noise scale

$$\sigma_b \approx 1.4826 \text{ median}(|p_{\partial} - b|). \quad (7)$$

Pixels are active when

$$p[u, v] > \max(b + 3\sigma_b, 10^{-2}p_{\max}). \quad (8)$$

Width and height are estimated independently, which is useful for motion blur or astigmatic PSFs. The proposal remains editable.

4 Graphical workflow

4.1 Tab 1: image and PSF

The main actions are ordered as: **Load disturbed image**, **Load reference image** (optional), **Load PSF**, **Generate test image**, **Generate selected PSF**, **Generate degraded input**, and **Clear images**. The last action removes all loaded/generated images, PSFs, reconstruction histories and results while preserving the current GUI and algorithm settings. For experimental data, loading the disturbed image creates only the reconstruction input and explicitly leaves the reference absent. An independent ground-truth image may be loaded separately; it is used only for PSNR, SSIM, and reference-based Auto criteria, never as the reconstruction input. Therefore applying thresholds in Tab 2 cannot create a reference image. Previews, histograms, and resolution labels describe the committed calculation data. If source image and PSF arrays have different dimensions, the smaller array is centered on a common zero-padded canvas; no resampling is performed by this synchronization.

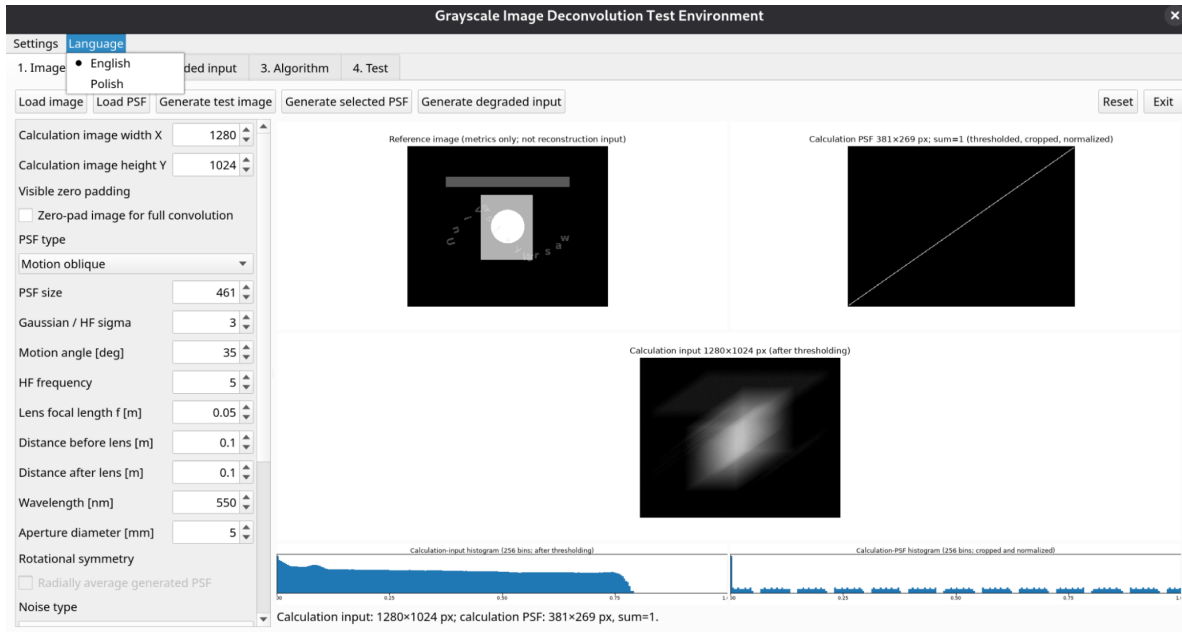


Figure 1: Main application window. Tab 1 shows the committed calculation image and PSF, their 256-bin histograms and numerical resolutions; the open Language menu documents the English/Polish GUI selector.

4.2 Tab 2: thresholds and calculation PSF

Two 256-bin histograms are located above the corresponding threshold controls. In full-array PSF preview, the area outside the committed rectangle is black because it is numerically zero. The compact preview shows the cropped, normalized kernel.

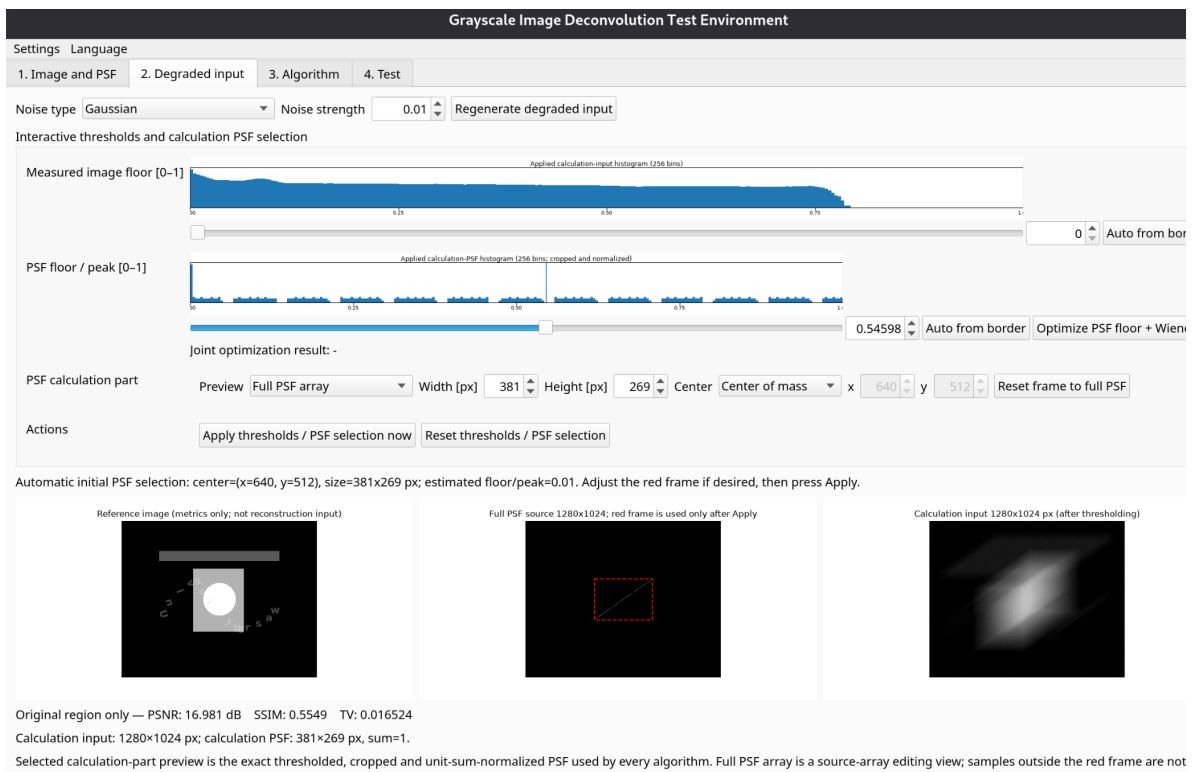


Figure 2: Preparation of the calculation PSF in Tab 2. The upper controls contain the applied histograms and pending thresholds; the red rectangle in the full-array preview defines the support that will be thresholded, cropped and normalized after Apply.

Optimize PSF floor + Wiener K searches for a plausible PSF floor and Wiener regularization. With a reference image it minimizes reconstruction MSE. Without a reference, the floor is constrained by robust PSF-background statistics and GCV selects K conditionally for each fixed PSF candidate. Kernels that collapse to a few samples or lose nearly all mass are rejected or penalized.

4.3 Tab 3: algorithm

Tab 3 selects the reconstruction method and parameters. Optional Wiener initialization, TV processing, and neural denoising are explicit. Auto tuning freezes their enabled/disabled state at the start, so parameters of disabled stages are not changed. Blind algorithms take the PSF width and height directly from the rectangle in Tab 2.

4.4 Tab 4: results

All stored iterations are assessed in batches, using Torch/CUDA where practical. After reconstruction the best iteration is selected and **Auto levels** is applied. Black and white controls span the full normalized display interval, maintain a positive gap, and alter visualization only. Cached histograms provide approximate percentiles without repeated full-image sorting.

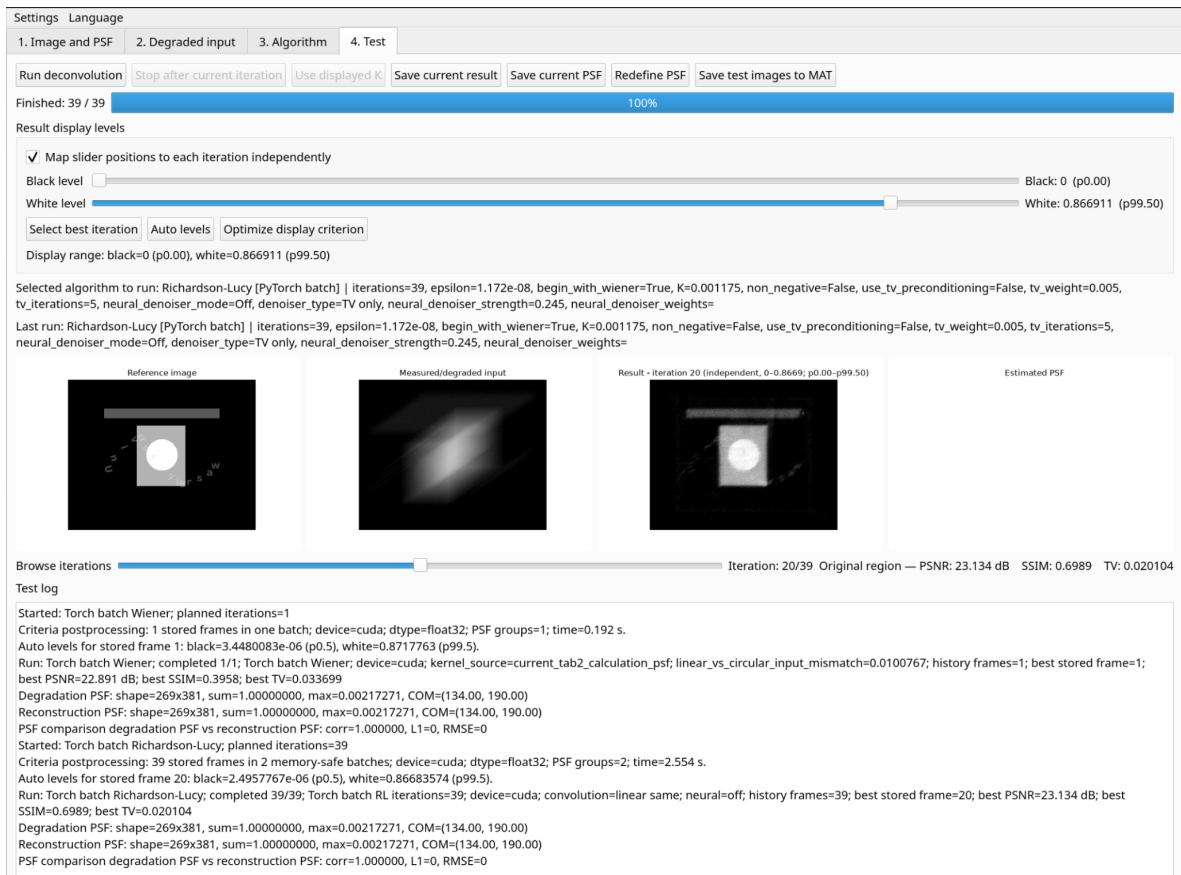


Figure 3: Iteration assessment in Tab 4 after a multi-iteration reconstruction. The selected frame, batch-computed quality criteria, display levels, reconstruction history and diagnostic log are visible.

References in the method sections point to original or classical sources where a clear source exists. Hybrid methods and stabilization choices specific to this program are explicitly identified as implementation-specific; citing their parent methods does not imply an exact reproduction of those publications.

5 Wiener filtering

Every Wiener stage uses explicit FFT and inverse FFT operations. The spectral minimum-mean-square-error construction follows the classical Wiener framework [1]. For scalar K and optional normalized noise spectrum $N[k, \ell]$,

$$\hat{F}[k, \ell] = \frac{H^*[k, \ell]G[k, \ell]}{|H[k, \ell]|^2 + KN[k, \ell]}, \quad (9)$$

In this equation, $H^*[k, \ell] = \overline{H[k, \ell]}$ is the elementwise complex conjugate of the PSF spectrum. It is also the diagonal Fourier-basis representation of the adjoint of circular convolution by h_0 . followed by

$$\hat{f} = \text{Re}\{\mathcal{F}^{-1}[\hat{F}]\}. \quad (10)$$

The obsolete magnitude-of-IFFT output was removed because it is nonlinear and converts negative ringing into positive intensity.

For fixed PSF, generalized cross-validation uses the leverage

$$L_H = \frac{|H|^2}{|H|^2 + KN}, \quad R_H = 1 - L_H, \quad (11)$$

and a normalized cost of the form

$$\text{GCV}(K) = \frac{\langle |R_H G_c|^2 \rangle / \langle |G_c|^2 \rangle}{\langle R_H \rangle^2}, \quad (12)$$

where G_c is the spectrum of the mean-centered measured image. The parameter-selection principle is related to generalized cross-validation [2]. GCV selects K for a fixed PSF; it is not used as unrestricted proof that a strongly thresholded PSF is physically correct.

6 Richardson–Lucy family

6.1 Classical Richardson–Lucy

The multiplicative update is

$$f^{(t+1)} = f^{(t)} \odot \mathcal{H}_{\text{lin}}^* \left[\frac{g}{\mathcal{H}_{\text{lin}} f^{(t)} + \varepsilon} \right]. \quad (13)$$

The update follows the independent formulations of Richardson and Lucy [3, 4]. The implementation uses the exact adjoint of the zero-boundary linear *same* operator. Initialization can be a constant image or a Wiener result. Nonnegativity, TV processing, and a neural denoiser can be applied.

6.2 Richardson–Lucy–Wiener

This experimental hybrid performs an RL update and then a Wiener operation in every iteration. It combines the parent Richardson–Lucy and Wiener methods [3, 4, 1]; it is a program-specific hybrid rather than a reproduction of a single original algorithm.

6.3 Richardson–Lucy–Rosen

The implementation keeps the Richardson–Lucy forward prediction and ratio, but replaces the adjoint back-projection factor in Eq. (13). At iteration t it first evaluates

$$r^{(t)} = \frac{g}{\mathcal{H}_{\text{lin}} f^{(t)} + \varepsilon}, \quad R^{(t)} = \mathcal{F}\{r^{(t)}\}.$$

The compact PSF is embedded and shifted to the FFT origin on the image-sized grid, producing h_0 and its spectrum $H = \mathcal{F}\{h_0\}$. Equation (14) is then used with $F_1 = R^{(t)}$ and $F_2 = H$:

$$C_{L,M}^{(t)} = (|R^{(t)}| + \varepsilon)^L (|H| + \varepsilon)^M \exp \left[i \left(\arg R^{(t)} - \arg H \right) \right]. \quad (14)$$

The inverse transform gives a real spatial correlation map,

$$c_{L,M}^{(t)} = \text{Re} \left\{ \mathcal{F}^{-1} \left[C_{L,M}^{(t)} \right] \right\}.$$

The code then applies a global offset removal and peak normalization,

$$b_{L,M}^{(t)} = \frac{c_{L,M}^{(t)} - \min c_{L,M}^{(t)}}{\max \left(\max(c_{L,M}^{(t)} - \min c_{L,M}^{(t)}), \varepsilon \right)},$$

and replaces the classical factor $\mathcal{H}_{\text{lin}}^* r^{(t)}$ in Eq. (13) by $b_{L,M}^{(t)}$. The actual multiplicative step is therefore

$$f^{(t+1)} = f^{(t)} \odot \left(b_{L,M}^{(t)} + \varepsilon \right),$$

followed, when enabled, by TV processing, neural denoising, and projection onto the nonnegative cone.

Ignoring ε and the spatial min–max normalization, the special case $L = M = 1$ gives

$$C_{1,1}^{(t)} = |R^{(t)}| |H| \exp \left[i(\arg R^{(t)} - \arg H) \right] = R^{(t)} H^*.$$

Thus it reduces to ordinary circular cross-correlation of the Richardson–Lucy ratio with the PSF. It is nevertheless *not exactly identical* to the classical update in Eq. (13): the latter uses the exact adjoint of the zero-boundary linear *same* operator, whereas the Rosen correction uses image-sized circular FFT correlation and is subsequently offset and normalized to the interval $[0, 1]$. Consequently, the GUI preset $L = M = 1$ aligns the nonlinear exponents with ordinary correlation, but does not remove these boundary-model and normalization differences.

Values $L, M < 1$ compress spectral magnitudes and increase the relative influence of phase. If relaxation toward one is enabled, the exponents are updated after each completed iteration according to

$$L_{t+1} = \rho L_t + (1 - \rho), \quad M_{t+1} = \rho M_t + (1 - \rho), \quad 0 \leq \rho \leq 1,$$

so they approach one for $\rho < 1$. The construction is inspired by nonlinear reconstruction and the Lucy–Richardson–Rosen literature [11, 12]; the exact circular correlation, min–max normalization, and relaxation controls are implementation-specific.

7 Landweber, Kaczmarz, and Adam methods

7.1 Landweber

The classical update is gradient descent on the least-squares data term, following Landweber’s iteration for first-kind integral equations [5]:

$$f^{(t+1)} = f^{(t)} + \alpha \mathcal{H}_{\text{lin}}^* (g - \mathcal{H}_{\text{lin}} f^{(t)}). \quad (15)$$

The Wiener-preconditioned version transforms the adjoint residual d as

$$\tilde{d} = \text{Re} \left\{ \mathcal{F}^{-1} \left[\frac{\mathcal{F}\{d\}}{|H|^2 + KN} \right] \right\} \quad (16)$$

and applies $f^{(t+1)} = f^{(t)} + \alpha \tilde{d}$.

7.2 Block Kaczmarz

Relation to the classical Kaczmarz method

Writing the linear deconvolution problem as $Af = g$, the classical row-action method introduced by Kaczmarz [6] updates one equation at a time. For row a_i^T it is

$$f^{(j+1)} = f^{(j)} + \omega \frac{g_i - a_i^T f^{(j)}}{\|a_i\|_2^2} a_i, \quad (17)$$

where ω is a relaxation factor. The update is an orthogonal projection for $\omega = 1$ and an under- or over-relaxed projection otherwise. A formal block version using a group of rows A_B is

$$f^+ = f + \omega A_B^\top (A_B A_B^\top + \eta I)^{-1} (g_B - A_B f), \quad (18)$$

with optional stabilization $\eta \geq 0$. For an image convolution problem, however, explicitly storing A or repeatedly solving the block normal equations is expensive.

Approximate ART-style implementation used in the program

The implemented method is therefore an *approximate block Kaczmarz/ART scheme*, not an exact algebraic block projection. Its blockwise image-reconstruction interpretation is related to algebraic reconstruction techniques (ART) [7]. Let $\mathcal{H}_{\text{lin}}$ be the zero-boundary linear convolution operator and let P_b select an observation block. At the start of each outer iteration the blurred estimate is evaluated once,

$$y = \mathcal{H}_{\text{lin}} f. \quad (19)$$

For every selected block b , the local residual is

$$r_b = P_b(g - y). \quad (20)$$

A separable smooth window W_b may be applied to reduce seams. The code uses a raised Hann window whose one-dimensional weights are $0.25 + 0.75w_{\text{Hann}}$, so boundary samples retain nonzero weight.

In the default stabilized full-sweep mode, all selected residual blocks are accumulated on the observation grid and divided pointwise by their accumulated coverage:

$$\tilde{r} = \frac{\sum_{b \in \mathcal{B}} P_b^\top (W_b \odot r_b)}{\sum_{b \in \mathcal{B}} P_b^\top W_b}, \quad (21)$$

where division is performed only where the denominator is positive. One global adjoint correction is then formed,

$$\Delta f = \omega \frac{\mathcal{H}_{\text{lin}}^* \tilde{r}}{\max(\sum_{u,v} h[u, v]^2, \varepsilon)}. \quad (22)$$

This coverage-normalized sweep retains the row-action idea—only selected observation blocks contribute—while suppressing brightness jumps and periodic block-grid artifacts that arose when every local correction was applied independently.

The optional legacy mode back-projects each block separately and retains only the corresponding reconstruction block. Overlapping contributions are normalized by the squared smooth-window weights. It is useful mainly for comparison because it is more sensitive to block boundaries.

Block geometry and ordering

The current implementation uses square observation blocks of side B . Without overlap the stride is B ; with overlap it is $\max(1, \lfloor B/2 \rfloor)$. Boundary starts are always included, so the complete image is covered. The user can choose:

- a full sweep over every block or only a specified number of blocks per outer iteration;
- deterministic cyclic order or a reproducible randomized order;
- overlapping or non-overlapping blocks;
- a shifted block lattice between full sweeps, which changes the block boundaries and reduces stationary grid artifacts;
- smooth block weighting or uniform weights.

Stability controls and optional processing

Before application, the raw update can be clipped relative to the current signal scale,

$$s = \max(Q_{0.99}(|f|), Q_{0.99}(|g|), 10^{-6}), \quad \Delta f \leftarrow \text{clip}(\Delta f, -\rho s, \rho s), \quad (23)$$

where ρ is the maximum update fraction. A damping parameter $\beta \in [0, 1]$ then gives

$$f^+ = (1 - \beta)f + \beta(f + \Delta f). \quad (24)$$

The estimate may subsequently be projected onto nonnegative values and passed through the optional TV preconditioner or neural denoiser. Wiener initialization is also available when explicitly enabled.

GUI parameter	Numerical role
Kaczmarz relaxation	ω , scale of the adjoint residual correction.
Kaczmarz block size	Side length B of square observation blocks.
Blocks per iteration	Number of blocks used when full sweep is disabled.
Full sweep	Visits all blocks in every outer iteration.
Overlapping blocks	Uses approximately 50% overlap in each direction.
Randomized blocks	Visits a random subset/order instead of deterministic cycling.
Shifted grid	Moves block boundaries between full sweeps.
Smooth block window	Applies raised-Hann weights to reduce seams.
Stabilized sweep	Combines coverage-normalized residuals before one global adjoint update.
Update damping	β , interpolation between the old and proposed estimate.
Max update fraction	ρ , clips an excessively large outer-iteration correction.

The method is most useful when blockwise data access, artifact diagnosis, or comparison with ART-style reconstruction is of interest. It should not be interpreted as an exact solution of the block projection equations, and its parameters are coupled: increasing overlap or the number of visited blocks generally requires reconsidering relaxation, damping, and the update limit.

Settings Language

1. Image and PSF 2. Degraded input 3. Algorithm 4. Test

Algorithm and parameters

Algorithm: Block Kaczmarz

PSF support used by selected method: Non-blind method uses only the current Tab 2 calculation PSF (381 × 269 px; sum=1), after thresholding, rectangular cropping and unit-sum normalization.

Wiener K: 0.00000012589

Iterations: 31

Kaczmarz relaxation: 0.052735

Kaczmarz block size: 50

Kaczmarz blocks per iteration: 16

Kaczmarz full sweep: ☒ Kaczmarz full sweep over all blocks each iteration

Kaczmarz overlapping blocks: ☒ Use overlapping Kaczmarz blocks

Kaczmarz randomized blocks: ☐ Visit Kaczmarz blocks in random order

Kaczmarz shifted grid: ☒ Shift Kaczmarz block grid between iterations

Kaczmarz smooth block window: ☒ Use smooth Kaczmarz block window

Kaczmarz stabilized sweep: ☒ Use stabilized Kaczmarz full-sweep update

Kaczmarz update damping: 0.554

Kaczmarz max update fraction: 0.094

Non-negative: ☒ Replace negative values with zero after each iteration

Begin with Wiener filter: ☐ Apply one Wiener filter before the selected algorithm

Wiener noise PSD: ☐ Use generated noise power spectrum in Wiener denominator

TV preconditioning: ☐ Apply a TV proximal step after each iteration

TV weight: 0

TV iterations: 5

Denoiser timing: Off

PyTorch backend: PyTorch: available; device: cuda

Auto optimization

Strategy: Full local grid (thorough)

Auto batch size: 16

Max batch size: 2048

GPU batch policy: Balanced (75%)

GPU benchmark: Detected CUDA batch: not benchmarked

Max candidates: 255

Numeric: ☒ Tune numeric parameters

Boolean: ☐ Tune boolean parameters

Figure 4: Block Kaczmarz controls in Tab 3: block geometry, sweep and ordering options, shifted grid, smooth window, stabilized update, damping and clipping parameters.

7.3 PyTorch Adam TV-MAP

The non-blind Adam method minimizes

$$J(f) = \frac{1}{2} \|\mathcal{H}_{\text{lin}} f - g\|_2^2 + \lambda_f \text{TV}(f) \quad (25)$$

with automatic differentiation. The TV prior follows the edge-preserving Rudin–Osher–Fatemi model [8], while parameter updates use Adam [9]. Candidate parameter sets can be processed in batches. Nonnegativity is enforced by projection when selected.

8 Blind deconvolution

8.1 Blind Richardson–Lucy

Blind RL alternates the image update with a PSF update obtained from correlation of the current image estimate and the relative-blur ratio. Alternating Richardson–Lucy blind deconvolution was described by Fish *et al.* [10]. The PSF is cropped to the Tab-2 dimensions, projected to nonnegative values, and normalized after each update. Initialization can use the current calculation PSF or a rectangular Gaussian. Optional rotational projection imposes symmetry.

8.2 Blind Adam TV-MAP

The blind PyTorch method jointly minimizes

$$J(f, h) = \frac{1}{2} \|h * f - g\|_2^2 + \lambda_f \text{TV}(f) + \lambda_h \text{TV}(h). \quad (26)$$

After every Adam step the PSF is projected onto $h \geq 0$ and $\sum h = 1$, with optional rotational symmetry. Separate learning rates are used for image and PSF. This is an implementation-specific joint MAP construction built from the TV and Adam components [8, 9], rather than a reproduction of one canonical blind-deconvolution paper.

Blind deconvolution is intrinsically non-unique. PSF width, image sharpness, noise, regularization, initialization, and stopping iteration can compensate for one another.

9 Metrics and best-iteration selection

With an independent reference, the program evaluates MSE, PSNR, and SSIM. For normalized peak value one,

$$\text{PSNR} = 10 \log_{10}(1/\text{MSE}). \quad (27)$$

Without a reference it uses a composite no-reference criterion including reblur consistency, intensity preservation, total variation, and residual whiteness. For candidate $\hat{f}$ the residual is

$$r = \mathcal{H}_{\text{lin}} \hat{f} - g. \quad (28)$$

Residual whiteness is estimated from short-range normalized autocorrelations of $r - \bar{r}$. Metrics for the history are computed in batches and cached, so returning to the best iteration does not rerun postprocessing.

10 Auto tuning and cancellation

Auto evaluates parameter candidates, using batched implementations when possible. Parameters belonging to optional stages that were disabled when tuning started are frozen. Auto calculations run in an isolated numerical process. **Cancel Auto** first requests cooperative stopping; if the current numerical iteration does not return within five seconds, the worker process is terminated while the Qt GUI process remains alive.

11 Batch processing and CUDA

PyTorch variants operate on stacks of candidates where possible. The key choices are: default `float32`; cached spectra for fixed PSFs; differentiable FFT convolution for changing PSFs; memory-limited chunking; grouping of blind-history frames by PSF shape; and CPU fallback when CUDA is unavailable or memory allocation fails. GPU acceleration is most useful for large images, many iterations, and candidate batches.

12 Python API and use without the GUI

The numerical implementation can be reused from scripts, notebooks, batch jobs, and other applications without constructing Qt widgets. The supported integration layer is `deconv.api`; it imports the core data models and algorithm registry, but does not import `PyQt5` or `legacy_runtime.py`. Canonical English algorithm identifiers are used independently of the selected GUI language.

12.1 Public objects and functions

Object or function	Purpose
GrayImage	Two-dimensional grayscale image; numerical values are stored in data .
PSF	Non-negative unit-sum point-spread function; the calculation kernel is stored in kernel .
DeconvolutionResult	Native result containing the final image, iteration history, metrics, and a textual summary.
as_gray_image, as_psf	Convert two-dimensional NumPy arrays to validated data models.
generate_test_image	Create the same synthetic test image as the GUI.
generate_gaussian_psf	Create the Gaussian PSF generated by the GUI.
generate_motion_psf	Create a horizontal or oblique motion PSF.
generate_high_frequency_psf	Create the oscillatory high-frequency test PSF.
generate_lens_incoherent_psf	Create the approximate incoherent thin-lens PSF.
generate_psf, available_psf_generators	Dispatch and enumerate all generated PSF families.
disturb_image	Apply the zero-boundary linear forward model and optional reproducible noise.
available_algorithms	List all 15 canonical CPU/Torch registry identifiers.
default_parameters	Return an independent copy of one algorithm's default parameters.
AutoTuneOptions	Configure the GUI-independent Auto search strategy and tunable parameter categories.
auto_tune_parameters	Select parameters and return scores, diagnostics, and optionally a final reconstruction.
auto_deconvolve	Select parameters and immediately run the accepted setting.
available_auto_strategies	List the supported quadratic and full_batched searches.
run_deconvolution	Run any registered algorithm with merged default and user parameters.
*_filter wrappers	Dedicated convenience functions for every CPU and Torch algorithm.
save_grayscale	Save a model or array as an 8-bit grayscale image.

A general call returns the native result object:

```
from deconv.api import run_deconvolution

result = run_deconvolution(
    measured_array, # two-dimensional values in [0, 1]
    psf_array, # non-negative two-dimensional kernel
    algorithm="Richardson-Lucy",
    iterations=40,
    epsilon=1e-8,
)
restored = result.image.data
history = [frame.data for frame in result.history]
```

The generic runner covers every method listed in the algorithm registry, including blind and Torch variants. For discoverability, dedicated wrappers are provided for Wiener, Richardson–Lucy, Richardson–Lucy–Wiener, Richardson–Lucy–Rosen, blind Richardson–Lucy, Landweber, Wiener-preconditioned Landweber, Block Kaczmarz, all five Torch batch methods, Adam TV-MAP, and blind Adam TV-MAP. The convenience wrappers expose the most frequently used parameters while accepting additional registered options as keyword arguments.

The PSF API likewise covers every PSF generated by the GUI:

```
from deconv.api import (
```

```

    generate_gaussian_psf, generate_motion_psf,
    generate_high_frequency_psf,
    generate_lens_incoherent_psf,
)

gaussian = generate_gaussian_psf(size=21, sigma=3.0)
motion = generate_motion_psf(size=31, angle_deg=35.0)
hf = generate_high_frequency_psf(
    size=31, frequency=5.0, sigma=4.0
)
lens = generate_lens_incoherent_psf(
    size=65, focal_length=0.05,
    distance_before=0.10, distance_after=0.10,
    wavelength=550e-9, aperture_diameter=0.005,
)

```

Arrays passed with the default `normalize_image=False` must already lie in $[0, 1]$; this avoids silent contrast stretching in quantitative workflows. Setting `normalize_image=True` requests min-max normalization. A supplied PSF is projected to non-negative values and normalized to unit sum. The same calculation-safe PSF preparation is used as in the GUI.

12.2 Automatic parameter selection

The parameter-selection logic used by the GUI is also available synchronously through the public API. A typical call is

```

from deconv.api import AutoTuneOptions, auto_deconvolve

tuning = auto_deconvolve(
    disturbed, psf,
    algorithm="Richardson-Lucy",
    reference=reference, # optional independent truth
    params={
        "iterations": 20,
        "epsilon": 1e-8,
        "begin_with_wiener": False,
    },
    auto_options=AutoTuneOptions(
        strategy="quadratic",
        use_torch_equivalent=True,
        tune_boolean=False,
        tune_categorical=False,
    ),
    progress_callback=print,
)
restored = tuning.deconvolution_result.image.data
print(tuning.best_params)

```

`auto_tune_parameters` returns an `AutoTuningResult` and performs no final run unless `run_best=True`; `auto_deconvolve` always executes the accepted parameter set. The returned object contains the requested and proxy algorithms, initial and accepted parameters, initial and final scores, candidate count, elapsed time, status text, the evaluated-candidate history, and the optional `DeconvolutionResult`.

When an independent `reference` is supplied, Auto maximizes the best PSNR found in each candidate's stored iteration history. Without a reference, ordinary Wiener tuning minimizes generalized cross-validation (GCV). The other methods maximize the negative of the same no-reference cost used by the GUI, which combines reblur consistency, normalized total variation, intensity preservation, and short-range residual whiteness.

Two strategies are exposed. The default `quadratic` strategy performs two short coordinate passes. Each numerical coordinate is evaluated below, at, and above the current value; when the three finite scores are locally concave, a parabola is fitted and its vertex is tested. Wiener K is handled separately by a broad logarithmic scan followed by two narrower log-space refinements. The `full_batched` strategy constructs a bounded local Cartesian pool and uses `run_batch` or `score_batch` when the selected implementation supports batched Auto.

As in the GUI, the activation state of optional processing stages is frozen before candidate generation. Thus a disabled Wiener initializer, denoiser, or optional TV preconditioner cannot be enabled temporarily, and its hidden dependent parameters remain unchanged. Numeric, independent Boolean, categorical, denoiser, TV, and K tuning for an already active Wiener initializer are controlled independently through `AutoTuneOptions`. When PyTorch is available, a plain algorithm can be tuned with its Torch-batched equivalent and then validated on the originally requested implementation. Setting `use_torch_equivalent=False` disables this proxy.

The synchronous API accepts a `progress_callback` and an optional `stop_event` with an `is_set()` method. Cancellation is checked between candidate evaluations. Unlike the GUI's isolated numerical worker, the API does not terminate the caller process if one individual FFT/CUDA iteration blocks for a long time.

12.3 Standalone examples

The repository contains several standalone programs. The Wiener example `examples/wiener_motion_blur.py` generates the standard test image, a 31×31 motion PSF rotated by 35° , a disturbed image with reproducible Gaussian noise, and a Wiener reconstruction:

```
from deconv.api import (
    generate_test_image, generate_motion_psf,
    disturb_image, wiener_filter,
)

reference = generate_test_image(width=384, height=256)
psf = generate_motion_psf(size=31, angle_deg=35.0)
disturbed = disturb_image(
    reference, psf, noise_sigma=0.01, seed=7
)
result = wiener_filter(
    disturbed, psf, K=2.0e-3, non_negative=True
)
restored = result.image.data
```

The complete program also saves individual PNG files and the comparison shown in Fig. 5. It can be run with

```
python examples/wiener_motion_blur.py \
    --output-dir wiener_motion_output
```

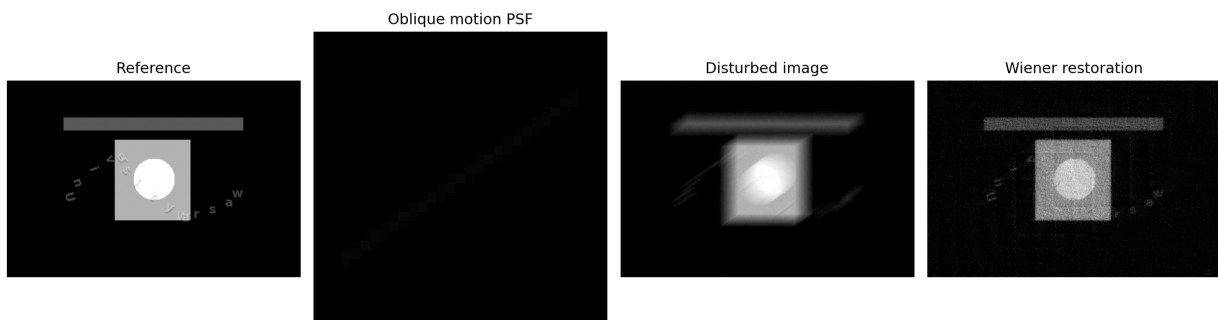


Figure 5: Output of the standalone API example: reference image, oblique motion PSF, disturbed input, and Wiener reconstruction.

Additional examples exercise other public wrappers and PSF families:

- `richardson_lucy_motion_blur.py`: Richardson–Lucy with an oblique motion PSF;
- `richardson_lucy_wiener_gaussian.py`: Richardson–Lucy–Wiener with a Gaussian PSF;
- `richardson_lucy_rosen_hf.py`: Richardson–Lucy–Rosen with a high-frequency PSF;
- `landweber_motion.py`: Landweber iteration with oblique motion blur;
- `kaczmarz_gaussian.py`: Block Kaczmarz with a Gaussian PSF;
- `auto_richardson_lucy_motion.py`: public Auto parameter selection followed by the accepted Richardson–Lucy reconstruction.

Each script saves the reference, PSF, disturbed input, reconstruction, and a comparison figure. These examples are intentionally simple and expose algorithm parameters directly, making them suitable starting points for custom batch-processing code.

12.4 Advanced execution hooks

Iterative methods accept `_iteration_callback` and `_stop_event`. The callback receives the current and total iteration numbers; a set event requests cooperative stopping after the current iteration. These underscored names are advanced execution hooks and may evolve more readily than the main API. GUI classes and application-state objects are deliberately excluded from the public numerical API.

13 Software architecture

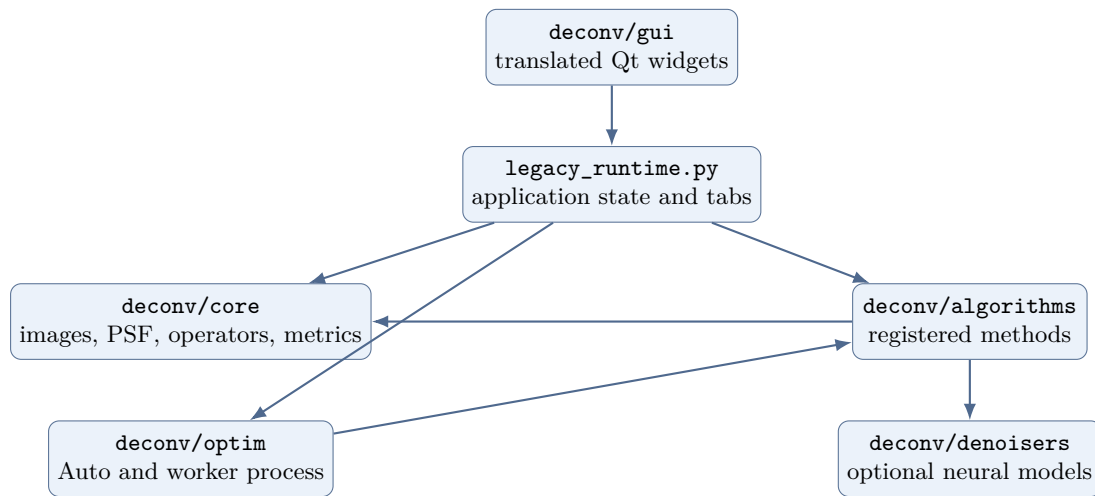


Figure 6: Main project modules and dependencies.

The GUI supports exactly English and Polish. English source strings are stable identifiers. Translated widget wrappers preserve canonical English algorithm names and configuration values. The selected language is stored in the active JSON profile; source comments, docstrings, and numerical identifiers remain English.

The default settings profile is stored in the user configuration directory. On Linux this is normally `~/.config/dekonwolucje`, unless `XDG_CONFIG_HOME` is set. The last profile and image directory are remembered.

14 Algorithm registry

GUI identifier	Role
Wiener	Closed-form circular FFT/IFFT inverse.
Richardson-Lucy	Multiplicative RL with exact linear-same adjoint.
Richardson-Lucy-Wiener	RL followed by Wiener filtering each iteration.
Richardson-Lucy-Rosen	RL-like nonlinear spectral-correlation variant.
Blind Richardson-Lucy	Alternating image and PSF updates.
Landweber	Least-squares gradient descent.
Landweber	Landweber with spectral gradient preconditioning.
Wiener-preconditioned	
Block Kaczmarz	Experimental block/ART residual projections.
Torch batch Wiener	Batched PyTorch/CUDA Wiener inverse.

GUI identifier	Role
Torch batch Richardson-Lucy	Batched RL.
Torch batch Richardson-Lucy-Wiener	Batched hybrid RL/Wiener.
Torch batch Richardson-Lucy-Rosen	Batched Rosen variant.
Torch batch Landweber	Batched Landweber.
PyTorch Adam TV-MAP	Non-blind differentiable MAP optimization.
PyTorch Blind Adam TV-MAP	Joint image/PSF MAP optimization.

15 Installation, testing, and extension

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
dekonwolucje
```

Alternative launch commands are `python -m deconv` and `python run_deconvolution_gui.py`. Optional PyTorch support can be installed with `python -m pip install -e ".[gpu]"`; a CUDA-compatible PyTorch wheel must match the local driver environment.

Tests are run with

```
python -m pip install -e ".[test]"
python -m pytest
```

To add a method, derive from the common algorithm interface, use shared data and operators, register the method, add canonical English controls and Polish translations, and add numerical and smoke tests.

16 Selected primary and classical references

References

- [1] N. Wiener, *Extrapolation, Interpolation, and Smoothing of Stationary Time Series: With Engineering Applications*, Technology Press of MIT and Wiley, 1949. doi:10.7551/mitpress/2946.001.0001.
- [2] G. H. Golub, M. Heath, and G. Wahba, “Generalized Cross-Validation as a Method for Choosing a Good Ridge Parameter,” *Technometrics* **21**, 215–223 (1979). doi:10.1080/00401706.1979.10489751.
- [3] W. H. Richardson, “Bayesian-Based Iterative Method of Image Restoration,” *Journal of the Optical Society of America* **62**, 55–59 (1972). doi:10.1364/JOSA.62.000055.
- [4] L. B. Lucy, “An iterative technique for the rectification of observed distributions,” *The Astronomical Journal* **79**, 745–754 (1974). doi:10.1086/111605.
- [5] L. Landweber, “An Iteration Formula for Fredholm Integral Equations of the First Kind,” *American Journal of Mathematics* **73**, 615–624 (1951). doi:10.2307/2372313.
- [6] S. Kaczmarz, “Angenäherte Auflösung von Systemen linearer Gleichungen,” *Bulletin International de l’Académie Polonaise des Sciences et des Lettres, Classe des Sciences Mathématiques et Naturelles, Série A* **35**, 355–357 (1937). An English translation is available at https://faculty.sites.iastate.edu/esweber/files/inline-files/kaczmarz_english_translation_1937.pdf.
- [7] R. Gordon, R. Bender, and G. T. Herman, “Algebraic Reconstruction Techniques (ART) for three-dimensional electron microscopy and x-ray photography,” *Journal of Theoretical Biology* **29**, 471–481 (1970). doi:10.1016/0022-5193(70)90109-8.
- [8] L. I. Rudin, S. Osher, and E. Fatemi, “Nonlinear total variation based noise removal algorithms,” *Physica D: Nonlinear Phenomena* **60**, 259–268 (1992). doi:10.1016/0167-2789(92)90242-F.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *International Conference on Learning Representations* (2015), <https://arxiv.org/abs/1412.6980>.
- [10] D. A. Fish, A. M. Brinicombe, E. R. Pike, and J. G. Walker, “Blind deconvolution by means of the Richardson–Lucy algorithm,” *Journal of the Optical Society of America A* **12**, 58–65 (1995). doi:10.1364/JOSAA.12.000058.
- [11] D. Smith *et al.*, “Nonlinear Reconstruction of Images from Patterns Generated by Deterministic or Random Optical Masks—Concepts and Review of Research,” *Journal of Imaging* **8**, 174 (2022). doi:10.3390/jimaging8060174.
- [12] V. Anand *et al.*, “Single-shot mid-infrared incoherent holography using Lucy–Richardson–Rosen algorithm,” *Opto-Electronic Science* **1**, 210006 (2022). doi:10.29026/oes.2022.210006.

17 Limitations and interpretation

- Wiener uses a circular model, while most iterative methods use zero-boundary linear convolution.
- Joint PSF-floor and K selection without a reference is heuristic; GCV selects regularization for a fixed PSF but does not uniquely identify the physical PSF.
- Blind methods are non-convex and depend on support, initialization, constraints, and stopping.
- Display levels alter visualization only, not stored reconstruction values.
- Fine reconstructed structure should be validated against independent measurements or simulations.

18 License and project information

The project is distributed under the MIT License.

Homepage: <https://github.com/rkotynski/dekonwolucje>

19 AI-assisted development disclosure

Parts of the software and documentation were prepared with assistance from tools based on large language models (LLMs). Their suggestions were incorporated as part of the development process. This assistance does not replace independent verification: numerical methods, implementation details, parameter choices, and reconstructed results should be validated for the intended scientific application.